

Diretrizes Obrigatórias do Projeto de POO (Tema Livre)

Objetivo: Desenvolver um sistema de terminal que, independentemente do tema escolhido, deve aplicar corretamente os quatro pilares da Programação Orientada a Objetos.

O sistema deve possuir um menu de terminal para interagir com as funcionalidades.

O projeto **obrigatoriamente** deve satisfazer os seguintes pontos:

1. Encapsulamento (Proteção dos Dados)

O estado interno dos seus objetos deve ser protegido e controlado.

- **Requisito 1.1:** Todos os atributos de suas classes de modelo (ex: `saldo` em `Conta`, `nome` em `Cliente`) devem ser declarados como `private`.
- **Requisito 1.2:** O acesso e a modificação desses atributos só podem ser feitos através de métodos públicos (**Getters e Setters**).
- **Requisito 1.3:** Pelo menos dois "Setters" no projeto devem conter uma **lógica de validação**. (Ex: `setSaldo` não pode permitir um valor negativo; `setEmail` deve verificar um formato básico; `setQuantidadeEstoque` não pode ser menor que zero).
- **Requisito 1.4:** As classes devem usar **Construtores** para garantir que um objeto já "nasça" em um estado válido, recebendo os dados essenciais para sua criação.

2. Herança (Reutilização e Especialização)

O projeto deve apresentar uma clara hierarquia de classes, onde conceitos genéricos são estendidos por conceitos específicos.

- **Requisito 2.1:** Deve existir uma **Classe Mãe (Superclasse)** que represente um conceito central e genérico do seu sistema (ex: `Pessoa`, `Veiculo`, `Conta`, `Personagem`).
- **Requisito 2.2:** A Superclasse deve definir atributos e métodos que são **comuns** a todos os seus descendentes (ex: `nome` e `cpf` em `Pessoa`; `velocidade` em `Veiculo`).
- **Requisito 2.3:** Devem existir pelo menos **duas Classes Filhas (Subclasses)** que **herdem** da Superclasse (ex: `Cliente` e `Funcionario` herdam de `Pessoa`; `Carro` e `Moto` herdam de `Veiculo`).
- **Requisito 2.4:** Cada Classe Filha deve ter **atributos e/ou métodos específicos** que a diferenciam da sua "irmã" e da sua mãe (ex: `limiteCredito` em `Cliente`; `salario` em `Funcionario`).

3. Abstração (O Essencial vs. a Complexidade)

O projeto deve focar em definir "o quê" um objeto faz, antes de se preocupar em "como" ele faz, e deve proibir a criação de objetos genéricos demais.

- **Requisito 3.1:** As suas **Classes Mãe (Superclasse)** principais (definida no Requisito 2.1) **devem ser declaradas como `abstract`**.
 - *Justificativa:* Isso impede que alguém crie um objeto "genérico" (ex: `new Pessoa()` ou `new Veiculo()`). Só será possível criar suas filhas específicas (`new Cliente()`, `new Carro()`).
- **Requisito 3.2:** A Classe Mãe Abstrata deve definir pelo menos **um método como `abstract`**.

- *Exemplos:* `public abstract void calcularTaxa();` (em `Conta`), `public abstract String exibirDetalhes();` (em `Produto`), `public abstract void atacar(Personagem alvo);` (em `Personagem`).
- *Justificativa:* Isso *força* todas as classes filhas a fornecerem sua própria implementação desse comportamento.

4. Polimorfismo (Muitas Formas, Uma Interface)

O sistema deve ser capaz de tratar objetos diferentes (mas que pertencem à mesma família) de maneira uniforme. Esta é a prova final de que a Herança e a Abstração foram feitas corretamente.

- **Requisito 4.1:** As Classes Filhas **devem obrigatoriamente sobrescrever (override)** o método abstrato definido na Classe Mãe (Requisito 3.2). Cada uma deve implementar o método de forma diferente, de acordo com sua especialidade.
- **Requisito 4.2 (O "Teste de Ouro"):** Deve existir uma "Classe Gerenciadora" (ex: `Banco`, `Loja`, `DungeonMaster`) que possua **uma única coleção (Lista ou Array)** da Superclasse Abstrata.
 - *Exemplo:* `List<Conta> contasDoBanco;` (e não `List<ContaCorrente>` e `List<ContaPoupanca>`).
 - *Exemplo:* `List<Personagem> inimigos;` (e não `List<Ogro>` e `List<Goblin>`).
- **Requisito 4.3:** Deve haver uma funcionalidade principal no menu (ex: "Listar Todos", "Aplicar Taxa Mensal", "Processar Turno de Ataque") que **itere sobre essa lista única** e chame o método sobrescrito (do Requisito 4.1) em cada item.
 - *Exemplo:* `for (Conta c : contasDoBanco) { c.calcularTaxa(); }`
 - O sistema deve executar o comportamento correto para cada tipo de objeto (a taxa da Conta Corrente e a da Poupança) **sem usar if/else ou switch para verificar o tipo do objeto** dentro do laço.

Funcionalidades Mínimas do Terminal

O menu do sistema deve ser construído em uma classe unificadores (Main.java) e permitir obrigatoriamente:

1. **Criar** objetos (ex: Adicionar um novo `Cliente`, criar um novo `Carro`).
2. **Listar** os objetos (Demonstrando o Polimorfismo - Requisito 4.3).
3. **Buscar** ou interagir com um objeto específico.
4. **Sair**.

Além de pelo menos 4 funcionalidades que serão inerentes ao projeto escolhido pelo grupo.

Desenvolvimento

O projeto deve ser inteiramente desenvolvido usando as ferramentas git e github, estando o repositório do projeto público e compartilhado com o usuário [silv4bufersa](#) para acompanhamento.

O acompanhamento do projeto será confirmado/comprovado através dos gráficos de contribuições do github, sendo a única opção aceita como prova de desenvolvimento do por parte dos integrantes.